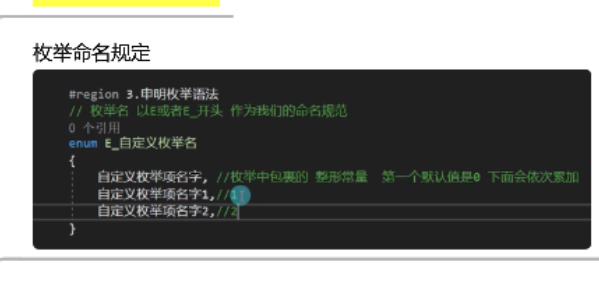
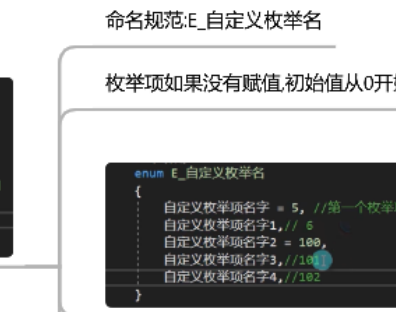
2.基础知识

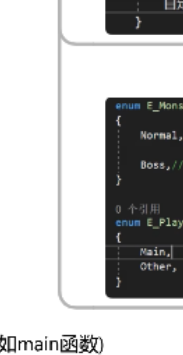
- 复杂数据类型变量
  - 枚举：整形常量的集合，可以自定义
  - 数组：任意变量类型顺序存储的数据
  - 结构体：任意变量的数据集合，可以自定义
- 枚举
  - 存储的是整型&&常量
  - 枚举命名规定



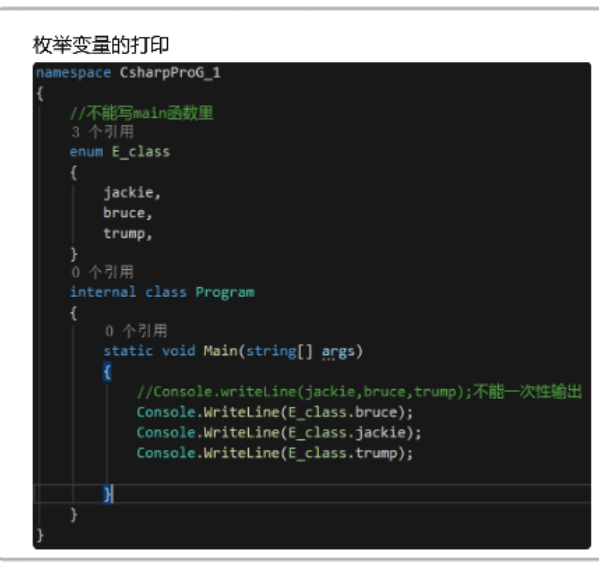
    - 命名规范 E_自定义枚举名
    - 枚举项如果没有赋值,初始值从0开始依次增加



    - 如果赋值了,下一项同样从该值开始累加
  - 枚举声明别

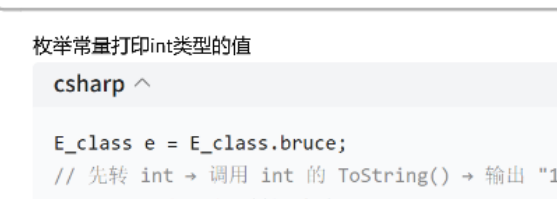


    - 注意枚举不能申明在函数中(比如main函数) 因为enum相当于上层结构不能声明在下层函数中
  - 枚举变量的打印



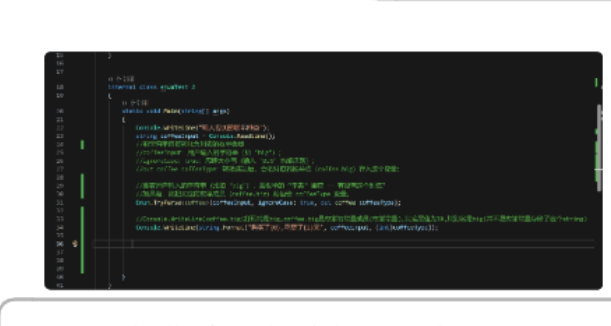
```
namespace CSharpProd_1
{
    //不能写在main函数里
    0 个引用
    enum E_class
    {
        jackie,
        bruce,
        trump,
    }
    0 个引用
    internal class Program
    {
        0 个引用
        static void Main(string[] args)
        {
            //Console.WriteLine(jackie,bruce,trump);不能一次性输出
            Console.WriteLine(E_class.bruce);
            Console.WriteLine(E_class.jackie);
            Console.WriteLine(E_class.trump);

        }
    }
}
```

    - 打印出的结果为string类型 和定义个int变量,打印出来直接是数字不同
    - 枚举常量相当于默认是string类型,只有强制转化为int类型(下层合题类)才能看见对应的整数值
  - 枚举常量打印int类型的值

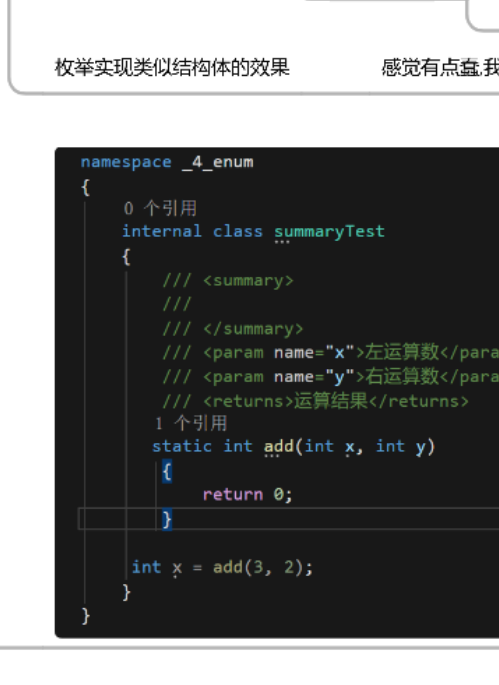


```csharp
E_class e = E_class.bruce;
// 实则 int = 调用 int 的 ToString() → 输出 "1"
Console.WriteLine((int)e);
```

    - 先将枚举变量接收成员常量,再将其强制转化
    - 好吧直接对成员常量强制转化也行
      - 强制转化并没有修改值本身,只是换了种类型对其进行读取
      - Console.WriteLine((int)E_class.bruce);
  - 字符串转化为枚举变量

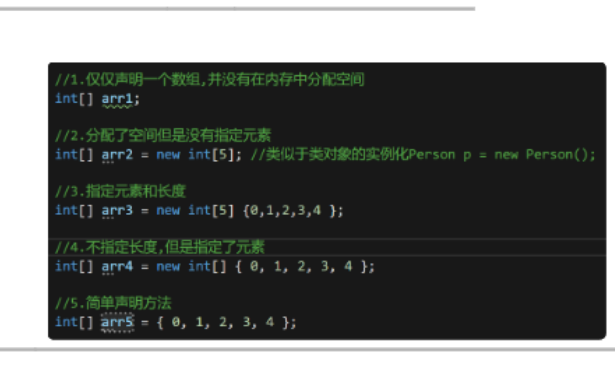


    - 如果是int类型转化为枚举变量,直接用(int)enum就可以了
    - 输入文字
      - 枚举本质是"数字 ↔ 字符名称"的双向映射表
      - 枚举变量相当于这一整行,而数字和名称相当于这一行中的两个对应元素
  - 枚举实现类似结构体的效果 感觉有点蠢,我选择跳过
- 注释

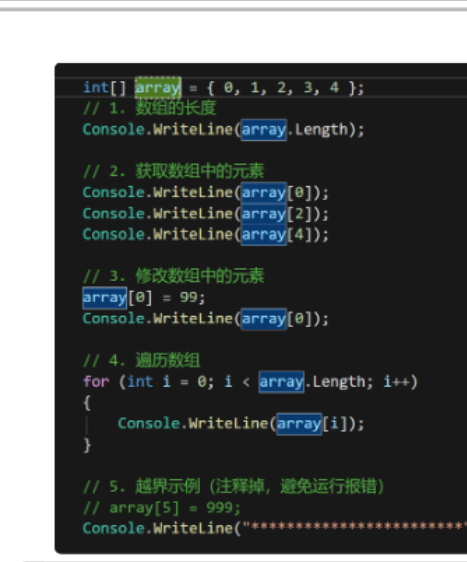


```
namespace _4_enum
{
    0 个引用
    internal class summaryTest
    {
        /// <summary>
        ///
        /// </summary>
        /// <param name="x">2个运算数</param>
        /// <param name="y">2个运算数</param>
        /// <returns>运算结果</returns>
        1 个引用
        static int add(int x, int y)
        {
            return 0;
        }

        int y = add(3, 2);
    }
}
```

  - 鼠标移动到函数上有解释
- 数组
  - 一维数组
    - 5种声明方式

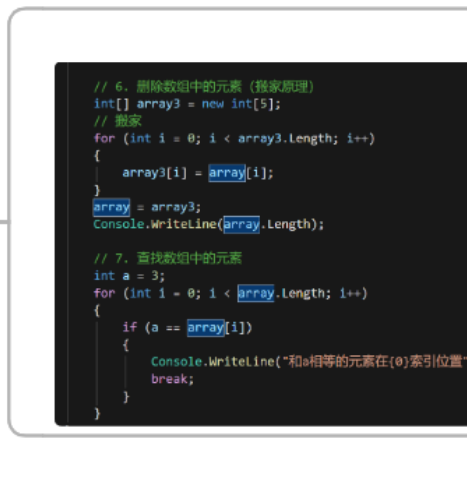


    - 简化声明仅适应指定长度

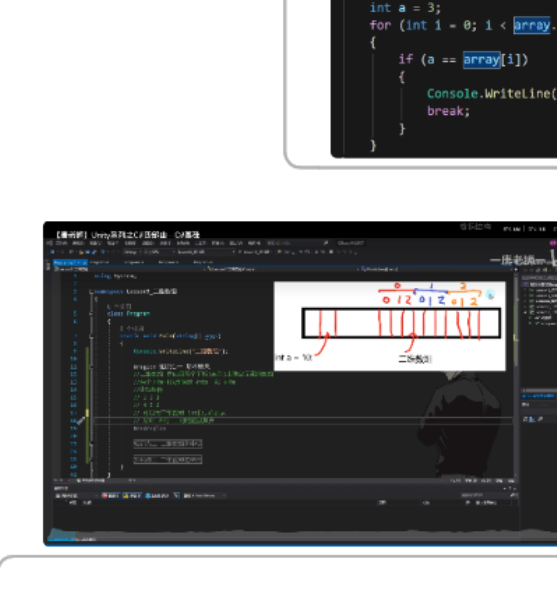


```
// 1. 获取数组的长度
Console.WriteLine(array.Length);

// 2. 获取数组中的元素
Console.WriteLine(array[0]);
Console.WriteLine(array[2]);
Console.WriteLine(array[4]);

// 3. 修改数组中的元素
array[0] = 99;
Console.WriteLine(array[0]);

// 4. 遍历数组
for (int i = 0; i < array.Length; i++)
{
    Console.WriteLine(array[i]);
}
```

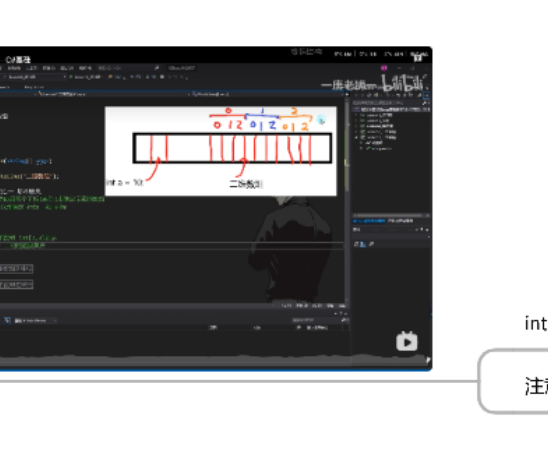
    - 一些有的没的基础操作



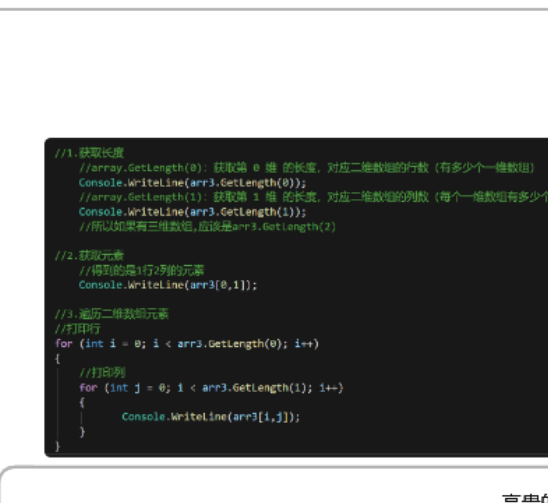
  - 二维数组

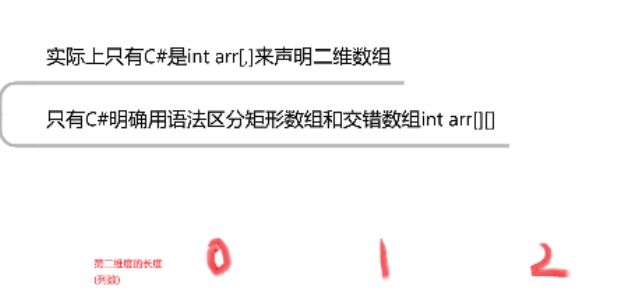


    - int arr[2,3]两行三列
    - 注意在内存中的存储方式
    - 几种声明

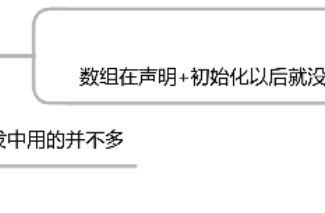


      - 实际上只有C#是int arr[,]来声明二维数组
      - 只有C#明确用语法区分矩形数组和交错数组int arr[][]
    - 使用方法

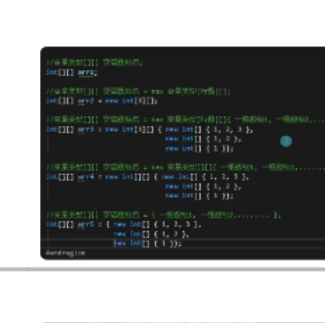




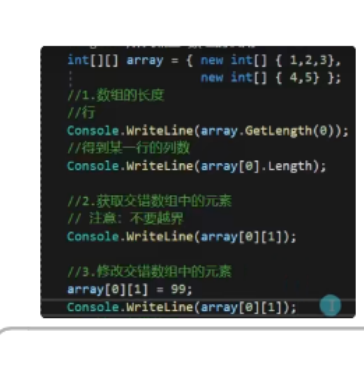
      - 数组在声明+初始化以后就没法继续修改了
        - 高贵的数据结构(顺序表,链表)了
        - 复制给另一数组,这里就不讲了
  - 交错数组
    - 实际的开发中用的并不多
    - 声明



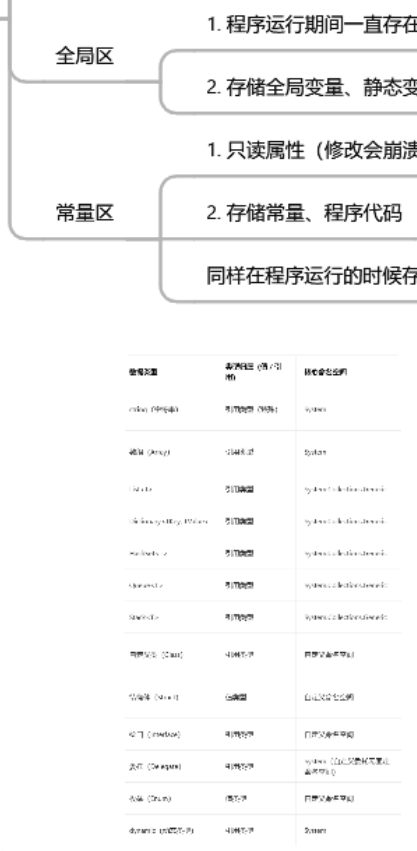
      - 每个子数组的类型要一样,如果这里有个float类型的数组会报错
      - 相当于数组的数组 和矩形数组区别就是不会指出列数,所以每行的个数(每个子数组的大小)可以不同
    - 使用



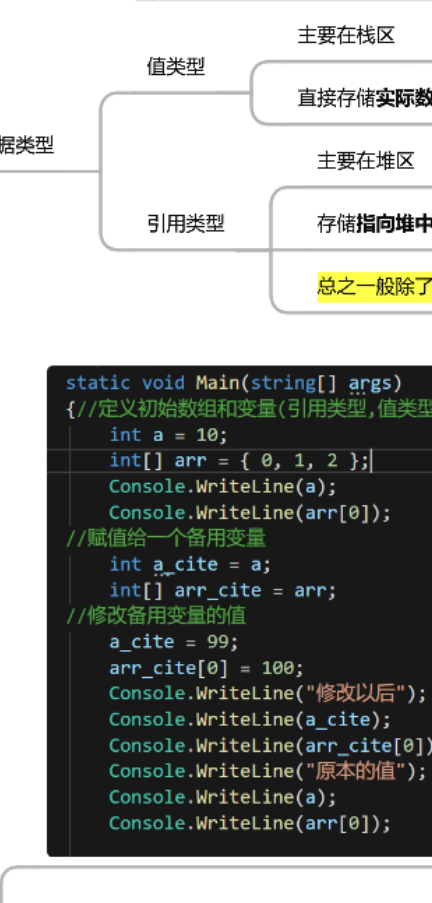
      - 获取到长度比较特殊,只能获取特定某一行的长度 所以相当于直接获取指定子数组的长度



      - 遍历只需要修改下内层循环
- 值类型和引用类型
  - 四种区(这里还是用C++举例吧)
    - 栈区
      - 局部变量、函数参数、返回值
      - 编译器自动分配 / 释放 随函数调用创建，函数结束立即释放
      - 数据按"栈"结构存储（先进后出）
    - 堆区
      - 堆就是完全二叉树(大根堆和小根堆)
      - 由程序员手动分配 / 释放（new/malloc分配、delete/free释放） 没有手动释放会导致内存泄露
    - 全局区
      - 1. 程序运行期间一直存在
      - 2. 存储全局变量、静态变量
    - 常量区
      - 1. 只读属性（修改会崩溃）
      - 2. 存储常量、程序代码 字符串常量（"hello"）、const常量（const int c_num = 30;）、函数指令
      - 同样在程序运行的时候存在
  - 数据类型

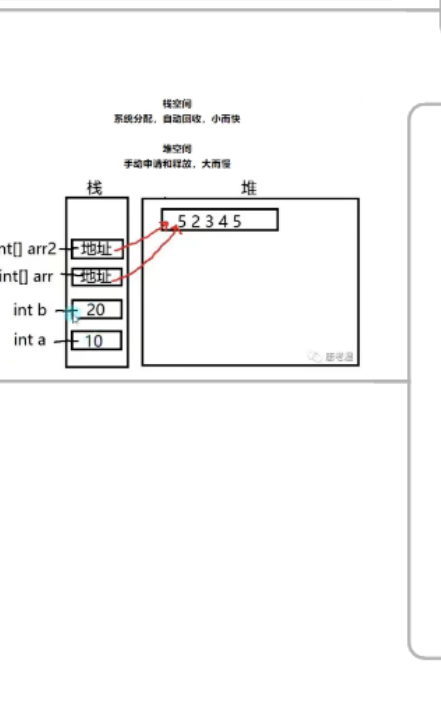


    - 简单/复杂数据类型 简单数据类型就是int,bool那些
    - 值/引用数据类型
      - 值类型
        - 主要在栈区 系统自动分配
        - 直接存储**实际数据值**
      - 引用类型
        - 主要在堆区 需要手动指定分配
        - 存储**指向堆中数据的内存地址**（引用 / 指针）
        - 总之一般除了类,数组,字符串这三种都是值类型(结构体也是值类型)
  - 值类型和引用类型的区别

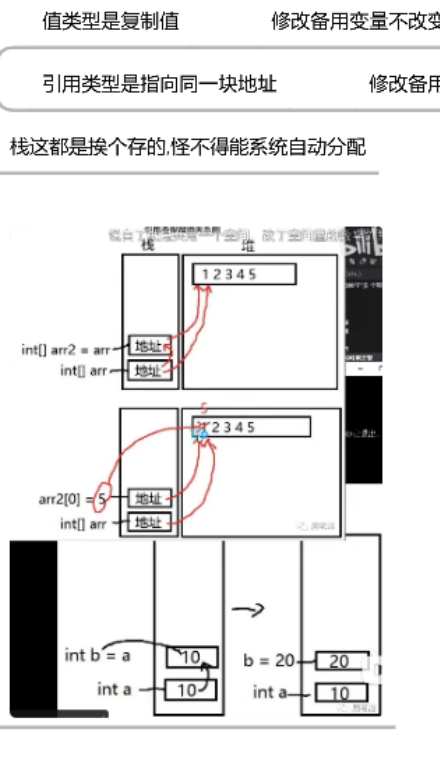


```
static void Main(string[] args)
{
    //定义初始数组和变量(引用类型,值类型)
    int a = 10;
    int[] arr = { 0, 1, 2 };
    Console.WriteLine(a);
    Console.WriteLine(arr[0]);
    //赋值给一个备用变量
    int a_cite = a;
    int[] arr_cite = arr;
    //修改备用变量的值
    a_cite = 99;
    arr_cite[0] = 100;
    Console.WriteLine("修改以后");
    Console.WriteLine(a_cite);
    Console.WriteLine(arr_cite[0]);
    Console.WriteLine("原本的值");
    Console.WriteLine(a);
    Console.WriteLine(arr[0]);
```

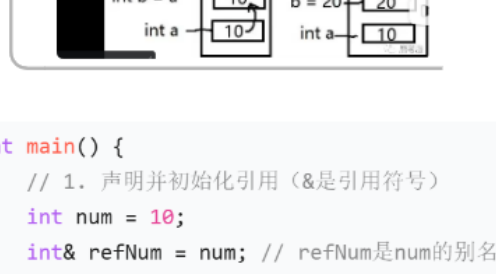
    - 值类型是复制值 修改备用变量不改变原有变量
    - 引用类型是指向同一块地址 修改备用变量,原有的变量也会变化



    - 栈这都是挨个存的,怪不得能系统自动分配

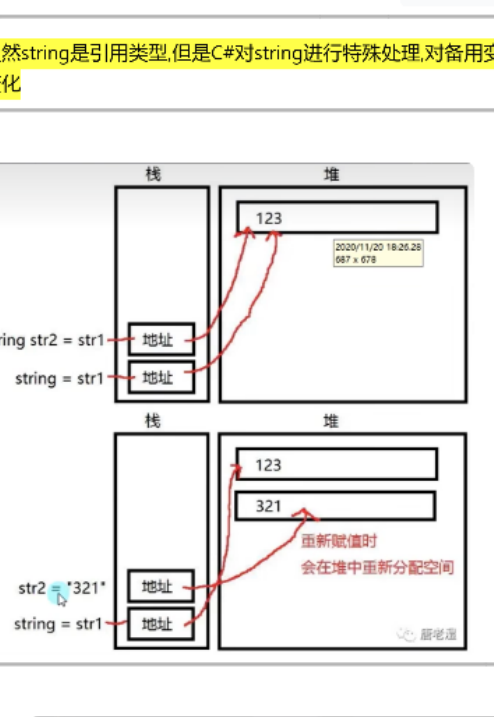


    - 对比C++的引用:为变量设置个别名



```
int main() {
    // 1. 声明并初始化引用（&是引用符号）
    int num = 10;
    int& refNum = num; // refNum是num的别名
```

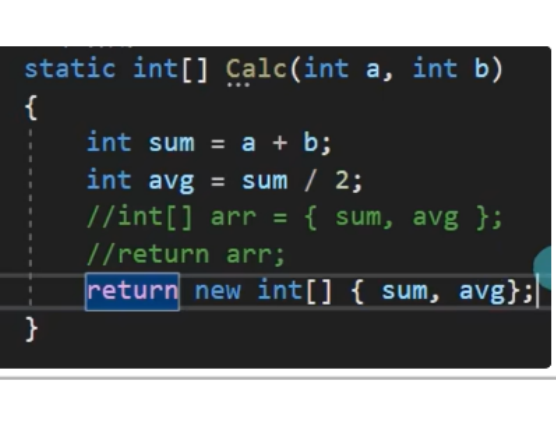
  - string特殊的引用类型
    - 虽然string是引用类型,但是C#对string进行特殊处理,对备用变量修改本体变量不会变化



    - 修改值的时候,在堆区中重新分配空间,而不是指向同一块内存
      - 注意只是声明个别名并不会重新分配空间
      - 所以频繁修改string的值会造成不必要的损耗
  - 监视引用变量和其指向内存

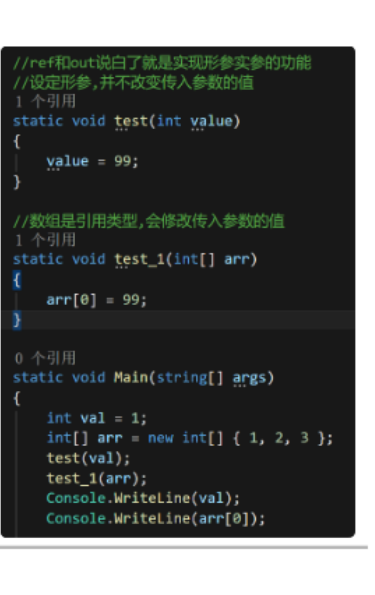
    - 左侧添加断点,调试中添加监视窗口
    - 输入变量名监视其值
    - 输入变量引用监视其指向内存
    - 输入变量解引用监视内存中的值
  - 引用类型使用前必须赋值(引用)
    - 输入文字
      - 仅仅Student stu;之后直接使用stu.name会报错 需要Student stu = new student();
      - 仅仅int[] arr;之后直接使用arr[0]变量会报错 需要 int arr = new int[3]; 当然也可以用简便方法赋予初始值
      - 仅仅string name;之后直接使用name变量会报错 字符串不需要new,只用赋予初始值
        - 实际上还是string s = new string('h','e','l','l','o');编译器帮忙简化了而已
    - 赋值null就不会报错,但并没有在堆区分配内存,只是做标记 Student stu = null;也是可以的
- 函数

  - 对比C++的函数
    - C#的不能在全局中任意声明,需要在class中(所以会说C#没有独立函数)
    - 还要加上private,public,static等修饰符
  - 函数命名规范
    - 驼峰命名法:myName
    - 帕斯卡命名法:MyName 函数用帕斯卡命名法
  - 有参有多个返回值的情况
    - C++函数不能直接返回多个返回值→一般是用结构体/指针/全局变量等来间接实现 那好吧,如果是相同类型返回值,同样可以用数组来实现(不过这样还是返回了一个数组吧)

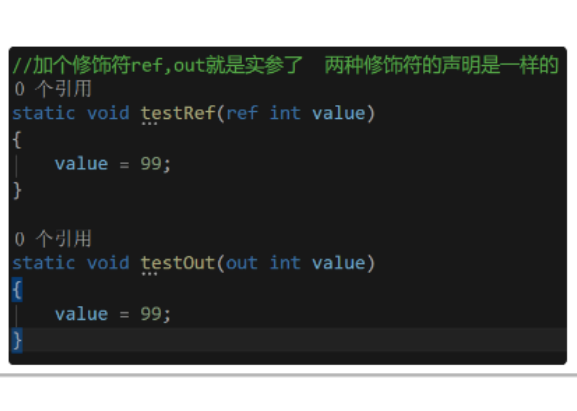


```
static int[] Calc(int a, int b)
{
    int sum = a + b;
    int avg = sum / 2;
    //int[] arr = { sum, avg };
    //return arr;
    return new int[] { sum, avg};
}
```

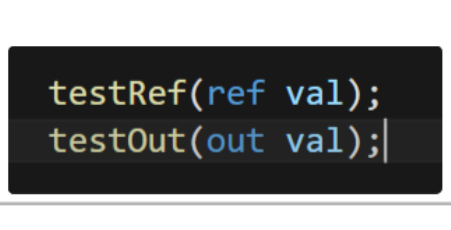
    - 数组实现返回相同类型值
- ref和out
  - 为啥使用



  - 使用

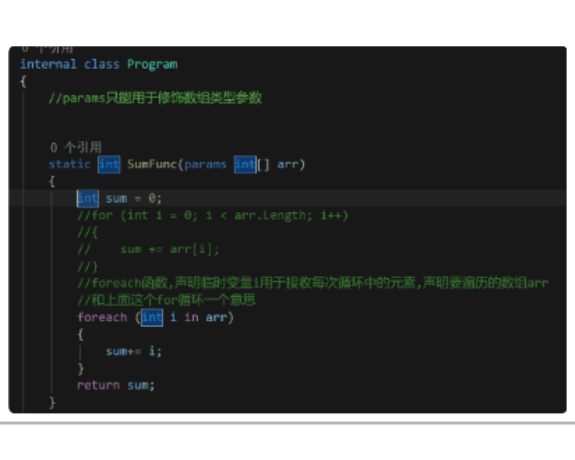


```
//加个修饰符ref,out就是实参了  两种修饰符的声明是一样的
0 个引用
static void testRef(ref int value)
{
    value = 99;
}
0 个引用
static void testOut(out int value)
{
    value = 99;
}
```

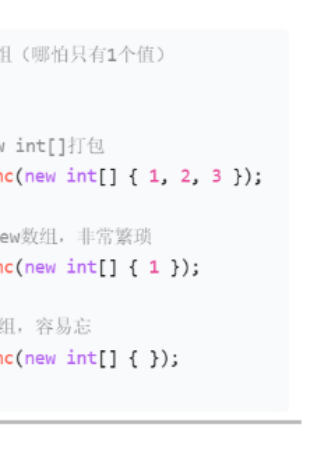


```
testRef(ref val);
testOut(out val);
```

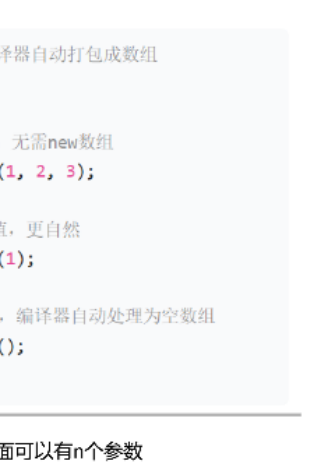
    - 调用函数的时候也需要加上修饰符
  - 区别
    - ref传入变量必须初始化,out不用 比如上面的传入val就要是有个初始值=3
    - out传入变量必须在内部赋值,ref不用 比如下面的传入val必须在内部改为99
- 变长参数
  - 比如求n个数之和,总不能每次都写int a~int d的参数吧,而且n的个数也是不确定的
  - 所以应该用到params关键字



    - params只能用于修饰数组类型参数

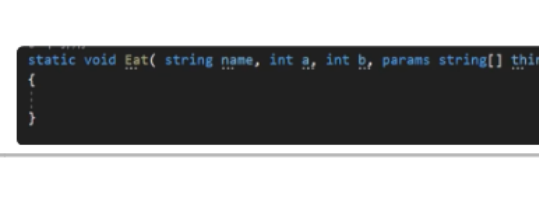


```
// 调用时：直接传参数组（编译器自动打包）
static void Main()
{
    // 求和1+2+3：直接传参个值，无需new int[]包裹
    int result1 = SumFunc(new int[] { 1, 2, 3 });

    // 只传1个：传数组
    int result2 = SumFunc(new int[] { 1 });

    // 不传参：需要传个数组，传空
    int result3 = SumFunc(new int[] { });
```

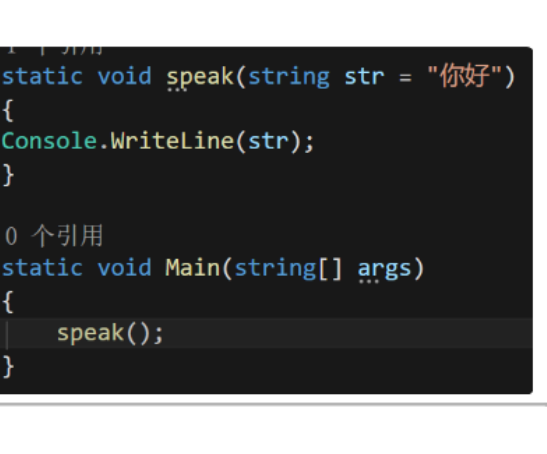


```
// 调用时：直接传参数组（编译器自动打包成数组）
static void Main()
{
    // 求和1+2+3：直接传参，无需new数组
    int result1 = SumFunc(1, 2, 3);

    // 只传1个：直接传单个值，更方便
    int result2 = SumFunc(1);

    // 不传参：也不会报错，编译器自动处理为空数组
    int result3 = SumFunc();
}
```

    - 直接传入数组也可以,但是每次调用传入参数时都要专门声明一个数组,麻烦
  - 混合使用

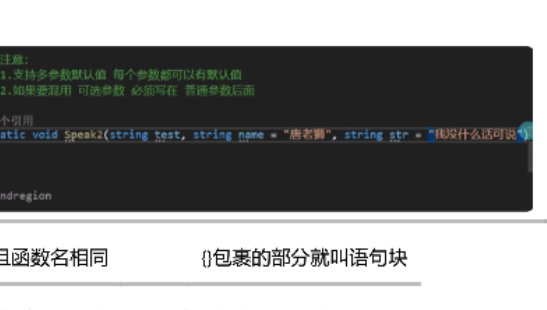


    - 但是参数中最多出现一个params参数,且需要再最后一组,前面可以有n个参数
    - 可以params修饰和普通参数混用
- 可选参数/参数默认值



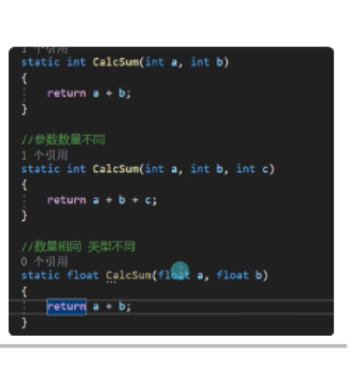
```
static void speak(string str = "你好")
{
Console.WriteLine(str);
}

0 个引用
static void Main(string[] args)
{
    speak();
}
```

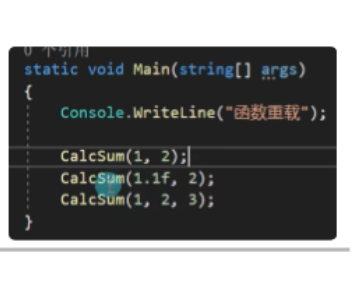
  - 声明函数时,参数已有默认值
  - 之后调用就可以不用再次传入参数了



  - 可以混用,同样要写在普通参数后面
  - 如果还有变长参数params,变长参数写在要后面
- 函数重载
  - 同一语句块中并且函数名相同 {}包裹的部分就叫语句块
  - 参数数量相同 或者 参数数量相同但是参数的类型和顺序不同
  - 比如WriteLine就是最经典的函数重载

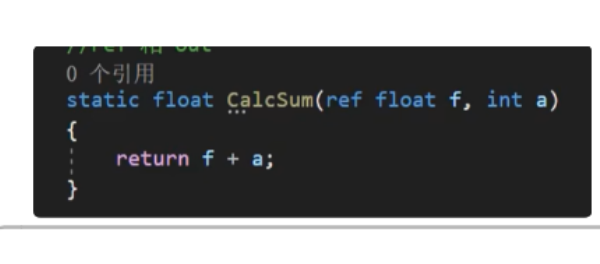
    - 同名函数但是支持一堆种类的输入
  - 使用
    - 重载和返回值类型无关,只看参数类型/个数/顺序
    - 调用时程序根据传入参数自动判断用哪个重载



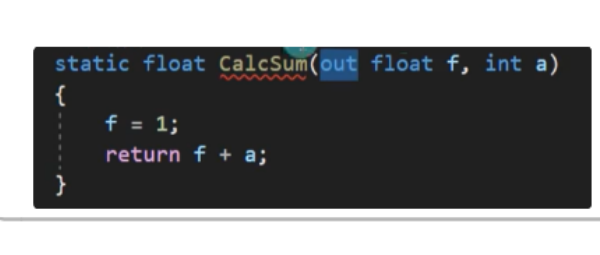


```
static void Main(string[] args)
{
    Console.WriteLine("函数重载");

    CalcSum(1, 2);
    CalcSum(1.1f, 2);
    CalcSum(1, 2, 3);
```

    - 形参实参带同样算重载

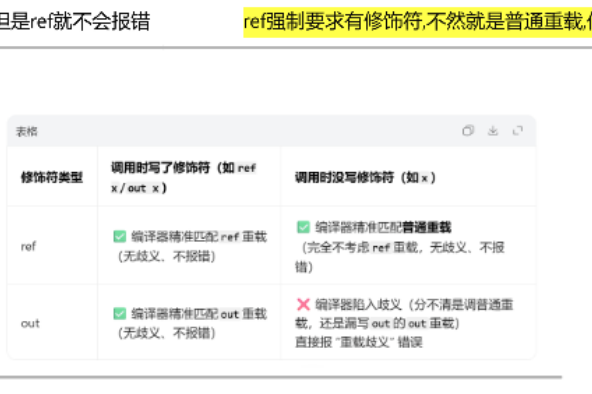


```
0 个引用
static float CalcSum(ref float f, int a)
{
    return f + a;
}
```



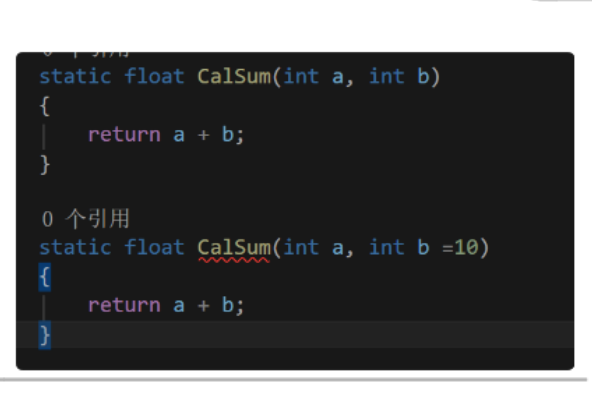
```
static float CalcSum(out float f, int a)
{
    f = 1;
    return f + a;
}
```

      - 只有今out参数是不同的,调用函数的时候编译器都不知道重载哪一个 重载out时候如果写的ShowNum(x)会报错
        - out如果设计成"没写修饰符就默认普通重载"只会引发更大的bug
        - 所以通过编译器报错来间接实现
      - 但是ref就不会报错 ref强制要求有修饰符,不然就是普通重载,但是out没有修饰符,两种重载都有可能



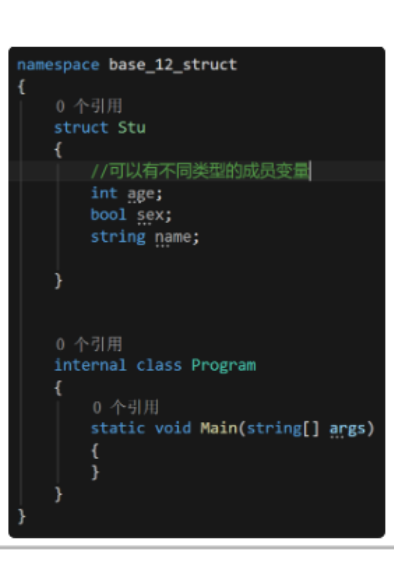
修饰符类型	调用时写了修饰符（如 ref x/out x）	调用时没写修饰符（如 x）
ref	✅ 编译器能判定是ref重载（无歧义、不报错）	✅ 编译器能判定是普通重载（因为不写ref就是普通传参，不报错）
out	✅ 编译器能判定是out重载（无歧义、不报错）	❌ 编译器陷入两难：①没写修饰符 ②out没写修饰符→无法判定该用普通重载还是out重载，直接报错

    - 只是设定了参数的初始值不能重载

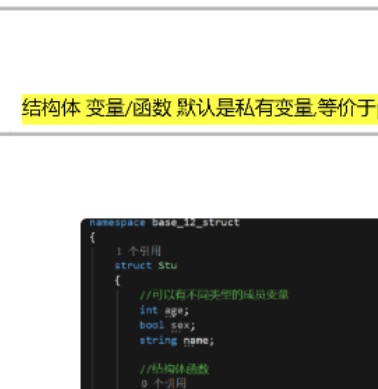


```
static float CalSum(int a, int b)
{
    return a + b;
}

0 个引用
static float CalSum(int a, int b =10)
{
    return a + b;
}
```

- 递归函数
  - 必须有终止调用的条件
- 结构体
  - C#里面函数要写在class里面,但是结构体又是写在namespace里面
  - 同样帕斯卡命名法

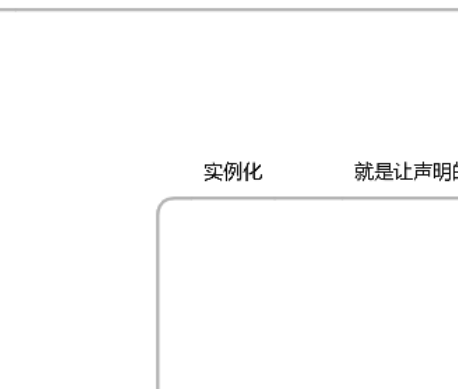


    - 结构体函数前面不用加static(这个还是面向对象的时候再看吧)
    - 结构体函数可以直接使用结构体内部的变量
  - 输入文字



```
namespace base_11_struct
{
    0 个引用
    struct Stu
    {
        //可以写不同类型的成员变量
        int age;
        bool sex;
        string name;
    }

    0 个引用
    internal class Program
    {
        0 个引用
        static void Main(string[] args)
        {
        }
    }
}
```

    - 注意结构体中的成员变量不能初始化
    - 成员变量可以是任意类型,包括结构体,唯独不能是自身的结构体 写个stu menba,就会报错
  - 访问修饰符
    - 结构体 变量/函数 默认是私有变量,等价于private int age
      - Stu后面也不会给你自动补齐,没法外部访问
      - 所以要写成public int age,这样可以用Stu.age来访问
  - 结构体的构造函数

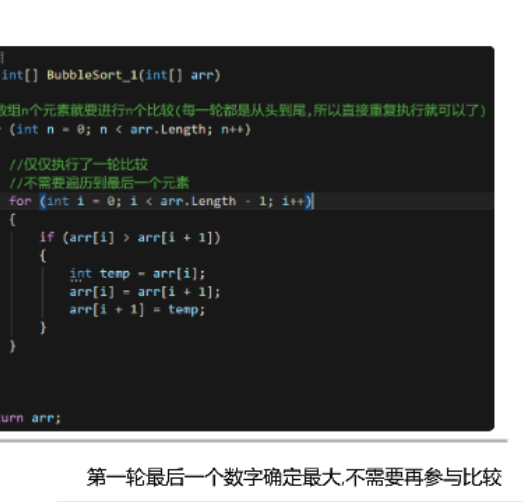


    - 要写在结构体中
      - 1.没有返回值
      - 2.函数名必须和结构体本身相同
      - 3.必须有参数 (我们这里是设定函数的成员变量作为参数)
      - 4.声明了构造函数,就必须对所有变量数据初始化 sex,name,age都得有,哪怕你乱写个初始值呢
  - 构造函数还有几种实例化这里串联一下
    - 实例化 就是让声明的结构体成员分配占用内存
    - 结构体的实例化(两种方法)
      - Stu jack;
        - 直接创建结构体对象,成员都是默认值 比如int = 0,string = null;
        - 调用之前必须手动给成员赋值才能访问实例 如jack.Name = "Jack"

      - Stu jack = new Stu();
        - 创建实例,成员同样均为默认值
        - 调用时可直接访问/修改成员 如jack.Age=20;

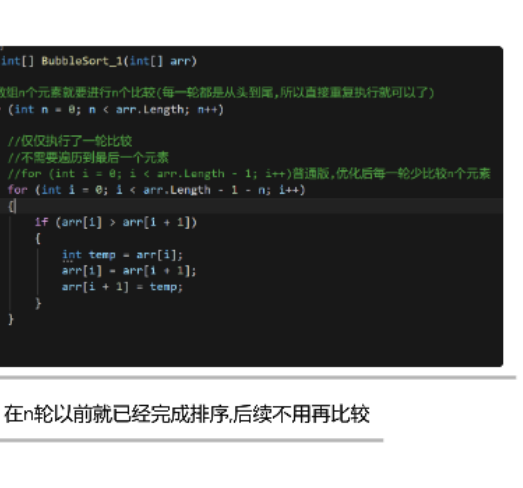
      - Stu jack = new Stu("Jack", 20);
        - 写了构造函数才能用
        - 创建实例时,直接给成员赋指定值,不需要后续赋值就可以用

        - 就是说对比第二种方法,可以少个jack.Age = 20; jack.Name = "Jack";
        - 省略步骤+强制初始化避免默认值bug
- 排序
  - 冒泡排序
    - 每一次比较都是和j+1来比较



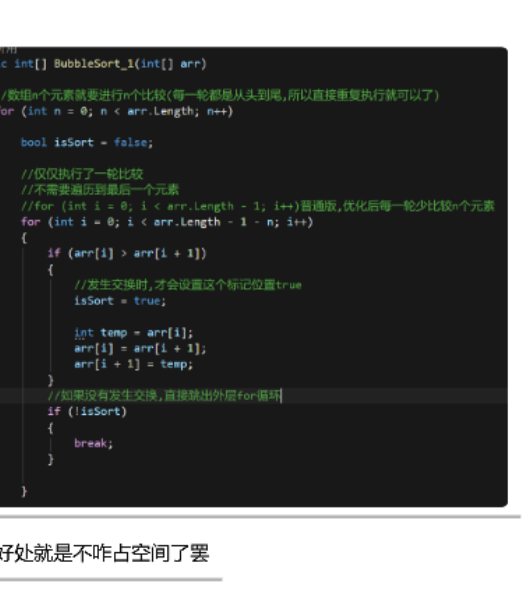
```
1 个引用
static int[] BubbleSort_1(int[] arr)
{
    //数组n个元素就要进行n个比较(每一轮都是从头到尾,所以直接重复执行就可以了)
    for (int n = 0; n < arr.Length; n++)
    {
        //仅仅执行了一轮比较
        //不需要遍历到最后一个元素
        for (int i = 0; i < arr.Length - 1; i++)
        {
            if (arr[i] > arr[i + 1])
            {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }

    }

    return arr;
```

    - 优化版
      - 第一轮最后一个数字确定最大,不需要再参与比较 第n轮最后n个数字确定最大



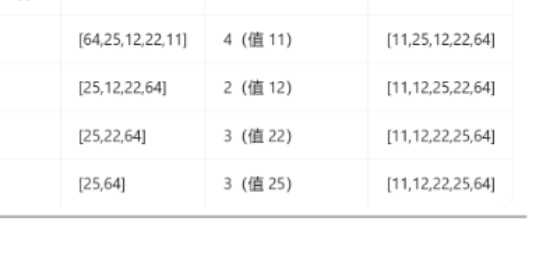
```
static int[] BubbleSort_1(int[] arr)
{
    //数组n个元素就要进行n个比较(每一轮都是从头到尾,所以直接重复执行就可以了)
    for (int n = 0; n < arr.Length; n++)
    {
        //仅仅执行了一轮比较
        //不需要遍历到最后一个元素
        //for (int i = 0; i < arr.Length - 1; i++)普通版,优化后每一轮少比较n个元素
        for (int i = 0; i < arr.Length - 1 - n; i++)
        {
            if (arr[i] > arr[i + 1])
            {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
    }
```

    - 进一步优化版
      - 在n轮以前就已经完成排序,后续不用再比较



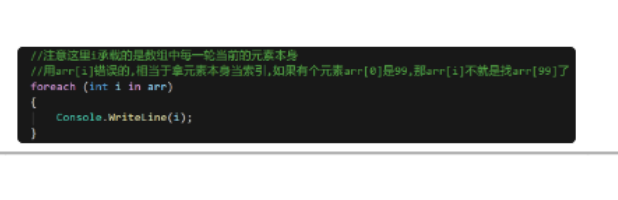
```
static int[] BubbleSort_1(int[] arr)
{
    //数组n个元素就要进行n个比较(每一轮都是从头到尾,所以直接重复执行就可以了)
    for (int n = 0; n < arr.Length; n++)
    {
        bool isSort = false;

        //仅仅执行了一轮比较
        //不需要遍历到最后一个元素
        //for (int i = 0; i < arr.Length - 1; i++)普通版,优化后每一轮少比较n个元素
        for (int i = 0; i < arr.Length - 1 - n; i++)
        {
            if (arr[i] > arr[i + 1])
            {
                //发生交换时,才会设置这个标记位置true
                isSort = true;

                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
            //如果没有发生交换,直接跳出外层for循环
            if (!isSort)
            {
                break;
            }
        }
```

  - 选择排序
    - 时间复杂度永远是n^2,唯一的好处就是不咋占空间了罢
    - 每一轮就是遍历找到最小的元素,然后把最小元素的放在首位 下一轮就不用看这个最小的元素了



轮次（i）	未排序区	找到的最小值索引	交换后数组
0	[64,25,12,22,11]	4（值 11）	[11,25,12,22,64]
1	[25,12,22,64]	2（值 12）	[11,12,25,22,64]
2	[25,22,64]	3（值 22）	[11,12,22,25,64]
3	[25,64]	3（值 25）	[11,12,22,25,64]

    - 输入文字
- foreach遍历补充



```
//这里i承载的是数组中每一轮当前的元素本身
//arr[i]错误的,相当于拿元素本身当索引,如果有个元素arr[0]是99,那arr[i]不就是找arr[99]了
foreach (int i in arr)
{
    Console.WriteLine(i);
}
```

  - 注意打印的是i而不是arr[i]
- C#基础贪吃蛇小游戏项目实现 输入文字